CHALMERS
UNIVERSITY OF TECHNOLOGY

UNIVERSITY OF GOTHENBURG

Feature-Based Product Derivation and Synchronization in Git

Design and evaluation of a feature-aware Git extension for derivation and synchronization

Bachelor of Science Thesis in Software Engineering and Management

Daniel Norberg
Erik Gabrielsson
Lilly Heier

Department of Computer Science and Engineering
UNIVERSITY OF GOTHENBURG
CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden 2026

CHALMERS
UNIVERSITY OF TECHNOLOGY

UNIVERSITY OF GOTHENBURG

Feature-Based Product Derivation in Git
Design and evaluation of a feature-aware Git extension for managing software product variants and synchronization

Supervisor: Daniel Strüber
Examiner: Sven Peldszus

University of Gothenburg
Chalmers University of Technology
Department of Computer Science and Engineering
SE-412 96 Göteborg
Sweden
Telephone + 46 (0)31-772 1000

Department of Computer Science and Engineering
UNIVERSITY OF GOTHENBURG
CHALMERS UNIVERSITY OF TECHNOLOGY
Gothenburg, Sweden 2026

Feature-Based Product Derivation and Synchronization in Git

Daniel Norberg
Software Engineering and Management
Chalmers and Göteborgs Universitet
Göteborg, Sweden
danno@chalmers.se

Erik Gabrielsson
Software Engineering and Management
Chalmers and Göteborgs Universitet
Göteborg, Sweden
erikgab@chalmers.se

Lilly Heier
Software Engineering and Management
Chalmers and Göteborgs Universitet
Göteborg, Sweden
lillyh@chalmers.se

***Abstract*—Git is the most widely used developer tool, yet it operates at the file and line level, which makes it difficult to manage feature-based code and product variants in software product lines. Developers who work on specific features or product variants must manually select files, keep track of which changes belong to which feature, and handle synchronization back to the main repository themselves. This thesis follows a Design Science Research methodology to design, implement, and evaluate an extension to Git that supports feature-based product derivation and feature-aware synchronization, building on an existing feature-oriented Git prototype developed at Ruhr-Universität Bochum. The extension introduces two new capabilities: automatic derivation of clean, ready-to-run product variants from a selected feature set, and provenance-based synchronization that maps edits made in a derived variant back to their original locations in the annotated source. The artifact is evaluated through a controlled, scenario-based correctness study covering all four edit categories of synchronization (equal-arity replacement, unequal-arity replacement, insertion and deletion) plus divergence detection, as well as a scalability measurement. The results show that the artifact produces structurally correct outputs across all tested scenarios and scales approximately linearly with the size of the repository.**

***Index Terms*—Feature-Oriented Software Development, Software Product Lines, Git, Version Control Systems, Feature Traceability, Product Derivation, Synchronization**

I. Introduction

Git is the most widely used version control system in modern software development [9]. Yet, it operates at the file and line level, which makes it unsuitable for feature-oriented software development. This thesis presents a Git extension that allows developers to automatically derive a ready-to-run product variant from a selected set of features and synchronize changes made in that variant back to the main repository without affecting unrelated features or annotation structure. The extension builds on an existing feature-oriented Git prototype developed at Ruhr-Universität Bochum [1] [2] and adds two new capabilities: feature-based product derivation and feature-aware synchronization.

In software product lines (SPLs), systems are built from combinations of features [8]. Git does not preserve explicit feature information, so feature implementations are scattered across branches or commits, and this information is typically lost during merges [3]. As a result, developers working on feature-specific code or product variants must manually select relevant files, track which changes belong to which feature, and carefully synchronize modifications back to the main repository. This process increases manual effort and creates structural risks: annotation markers can be accidentally removed, unrelated feature code can be disturbed, and edits can be applied at the wrong source positions.

To understand why these capabilities are needed, consider a system such as Word, with features like `AutoSave` and `SpellCheck`, whose implementations are interwoven with regular application logic inside the same file. The following example is pseudocode, intended to demonstrate how a file with multiple features may look; it does not use the annotation syntax introduced later in the thesis:

```
public void saveDocument(Document doc) {
    logSaveOperation(doc);    // base
    backupDocument(doc);      // AutoSave
    writeToDisk(doc);         // base
    checkSpelling(doc);       // SpellCheck
    updateRecentFiles(doc);   // base
}
```

If a developer wants to create a product variant with only the `AutoSave` feature, they have to manually identify all `AutoSave`-related code, exclude the `SpellCheck` block without accidentally removing surrounding logic, and construct a working variant by hand. After modifying this variant, they must then manually reapply those changes back to the main repository, taking care not to disturb the `SpellCheck` block or any annotation markers. Git provides no support for any of these steps.

This leads to three core problems:

1) locating feature-specific code,
2) deriving valid product variants, and
3) synchronizing the changes back without affecting other features.

The existing prototype by Röthemeyer [1] and Nguyen [2] addresses the first problem by introducing explicit feature

annotations and Git-native commands for managing feature associations. However, it stops at labeling and tracking. The developer still has to manually derive variants and manually reintegrate changes, which is error-prone and becomes increasingly difficult as the number of features grows.

This thesis addresses problems two and three by extending the prototype with:

1) product derivation, which automatically generates a clean, ready-to-run variant from a selected feature set by stripping annotation markers and non-selected feature code
2) feature-aware synchronization, which maps changes made in the derived variant back to the correct annotated locations in the main repository, preserving all unrelated feature content and annotation structure.

With the proposed extension, the developer can run a single command to derive a working `AutoSave` variant:

```
git feature project \
    --include AutoSave \
    --branch autosave-feature
```

After making changes, they run:

```
git feature project sync autosave-feature \
    --target main
```

The changes are then applied correctly to the `AutoSave` lines in the annotated source, leaving the `SpellCheck` block and all annotation markers completely untouched.

The artifact is evaluated through a controlled correctness study using 10 deterministic scenarios that cover all structural cases: single features, nested features, non-adjacent feature blocks, file-level exclusion, all four edit categories, and divergence detection. A scalability measurement on synthetic repositories of increasing size complements the correctness study.

The rest of this paper is structured as follows. Section II reviews related work on feature traceability, multidimensional editing, and feature-aware Git extensions. Section III describes the Design Science Research methodology and research questions. Sections IV, V, and VI present the three DSR cycles covering requirements, design and implementation, and evaluation, respectively. Section VII discusses the results and limitations, and Section VIII concludes the thesis.

II. Related Work

This section presents foundational concepts in feature-oriented software product lines and version control systems, then reviews prior research in three areas: (1) feature traceability and location, (2) multidimensional editing for managing variability, and (3) feature-aware extensions of Git. Together, these works establish the background for feature-based development while highlighting the limitations regarding product derivation and synchronization that this thesis addresses.

A. Foundations of Feature-Oriented Development and Version Control

A software product line (SPL) is a set of related software products that share common functionality but differ in certain aspects [8]. Instead of developing each product individually, an SPL approach reuses shared code while allowing variation between products. Systems are constructed from reusable parts rather than from scratch [8]. The differences between products in an SPL are described in terms of features, where each feature represents a unit of functionality that can be selected to produce a specific variant. For example, in a text editor product line, features could include spell-checking and autosave, and different products are achieved by selecting different combinations of these features [8]. Apel et al. state that feature orientation's primary goal is to structure all software artifacts and the entire product-line process around features [8], making it easy to link a customer's requirements to the software components that provide the relevant functionality.

Version control systems (VCS) are essential tools in modern software development. They enable developers to track changes and facilitate collaboration on joint projects [8]. Git is one of the most widely used version control systems in the world [9]. Git stores the project history locally and represents the different states of a repository as snapshots of files. Each commit records changes such as additions, deletions, or modifications, is accompanied by a unique hash, and serves as a specific snapshot of the project at that time [9]. Developers typically work on different versions using branches and merge to the main branch when the code is approved and finished, as the main branch usually consists of only stable code [9].

Merging is typically done at the line level, automatically using textual patches, and manual intervention is needed only when different changes affect the same line [8]. This design is not feature-aware: it tracks where lines change but not what feature they belong to. While this works well with traditional modular development, it does not support feature-oriented development. When feature branches are merged to main, feature information is typically lost [8]. Since features are mostly implemented using branches or scattered commits, feature information is not preserved [8]. This motivates the need for feature-aware extensions to Git, which the following sections review.

Given that traditional Git does not preserve explicit feature information, researchers have sought methods to improve feature traceability within software systems. A significant line of work, therefore, focuses on identifying, documenting, and maintaining explicit links between features and their corresponding code artifacts.

B. Feature-to-Code Mapping

A central challenge in feature-oriented development is maintaining explicit mappings between features and their corresponding implementation artifacts. In software product lines, features are often scattered across multiple files and commits, making it difficult to understand their implementation and evolution over time.

Several approaches have focused on improving feature traceability through feature associations. Schwarz et al. propose proactive feature annotations embedded directly in source code [6]. This approach enables developers to explicitly associate code fragments with features during development, thereby strengthening traceability and reducing ambiguity. Building on this idea, Röthemeyer [1] proposes an extension to Git that stores feature-related metadata and provides Git-native commands for managing feature associations. Nguyen [2] further improves and evaluates this prototype, demonstrating that explicit feature labeling enhances traceability and developers' understanding of feature-related changes. FLOrIDA supports automated feature location by extracting and visualizing feature traces in software product lines [5]. Helping developers identify where features are implemented improves feature comprehension and supports maintenance activities. FeatRacer extends this idea by combining annotations with machine-learning techniques to detect missing feature associations during commits, with evaluations across multiple open-source projects that demonstrate improved precision and recall compared to traditional feature-location techniques [7].

These approaches [1], [2], [5]–[7] primarily focus on identifying, documenting, or tracking features rather than enabling developers to derive product variants or synchronize feature-specific changes back into the shared repository.

The feature-oriented Git prototype by Röthemeyer [1] and its subsequent improvements by Nguyen [2] adopt a block-based annotation syntax originally inspired by FAXE (Feature Annotation eXtraction Engine, a library for parsing andretrievingg annotations [6]). In this syntax, feature code is enclosed between matching markers placed on their own lines:

```
# &begin[FeatureA]

doSomething();
doSomethingElse();

# &end[FeatureA]
```

The markers are language-agnostic; the comment prefix adapts to the file type (e.g., `#`, `//`). The syntax supports nested annotations and is used to explicitly link code regions to features. This annotation mechanism is the foundation on which the present thesis is built. So, although the prototype allows for tracking and annotating features, it does not provide mechanisms for derivation or synchronization.

C. Multidimensional Editing and Variant Management

Managing multiple software variants within a unified codebase has been explored using multidimensional editing systems. Kruskal's P-EDIT is an early approach in which lines of code are associated with Boolean expressions that represent different concerns [4]. This enables multiple variants to coexist within a single representation and allows developers to view and edit concern-specific projections of the code base. Conceptually, multidimensional editing aligns with feature-based product derivation, as both aim to generate variant-specific views from a shared underlying system.

However, P-EDIT operates at a fine-grained textual level and lacks integration with modern distributed version control systems such as Git. As a result, it does not address collaborative development scenarios where multiple developers work on evolving product variants. Walkingshaw and Ostermann generalize the projectional approach in their work on projectional editing of variational software, where developers edit projections of a variational codebase, and changes are lifted back into the shared source [11]. A central principle from that work, edit isolation, states that an edit made in one variant projection must not affect any other variant. This principle directly motivates the synchronization design adopted in this thesis. Stănciulescu et al. take the projectional idea further and propose a projection-based variation control system in which developers check out a variant, edit it as ordinary code, and have edits propagated back into a shared, annotated repository [14]. They establish the conceptual operations (project, edit, commit-back) and demonstrate feasibility on a small case study. Still, the system is not integrated with a mainstream version-control tool, and is not based on Git. Hinterreiter et al. introduce feature-oriented clone and pull operations that allow developers to retrieve selected features rather than entire repositories [3], improving reuse and reducing unnecessary integration effort compared to conventional workflows. Although these approaches support selective retrieval, projection, or commit-back at a conceptual level, none provide a Git round-trip in which a developer can derive a clean variant, edit it, and synchronize the edits back into an annotated source while preserving annotation markers and unrelated feature content. This thesis addresses that gap.

D. Feature-Aware Synchronization in Git

The projectional approaches discussed in Section II-C [4], [11], [14] establish the conceptual foundation for editing a variant in isolation and lifting changes back into a shared, annotated source. None of them, however, is integrated with Git, which is the version-control system in which the vast majority of modern software development takes place [9]. The feature-oriented Git extensions surveyed in Section II-B [1]–[3] are integrated with Git but address only labeling, tracking, and selective retrieval; they do not derive standalone product variants and do not propagate edits made in such variants back into the annotated source.

The consequence is that, in practice, a developer who wishes to work on a single feature in a Git repository has no tool support for the round trip. Edits made on a feature branch are integrated through standard line-level merges, which are unaware of feature boundaries and may overwrite annotation markers or disturb unrelated feature code. The problem is significantly more challenging when the developer works on a projected variant in which non-selected feature regions and annotation markers have been stripped, because the projected file and the annotated source no longer share a line-level coordinate system.

E. Research Gap

While prior research has substantially improved feature traceability through inline annotations, automated feature location, and feature-aware Git operations [1]–[3], [5]–[7], and has separately established the conceptual model of projection-based editing with edit isolation and commit-back [4], [11], [14], no existing approach identified in this study brings these two together. Specifically, no surveyed tool supports the full round-trip workflow of deriving a product variant from a selected feature set, editing that variant in isolation, and propagating the resulting edits back to the original annotated source, all within a standard Git workflow.

Deriving a variant requires stripping annotation markers and excluding non-selected feature blocks, which produces a clean, standalone codebase. However, this transformation introduces a structural gap: the derived file has fewer lines, no markers, and a shifted line numbering relative to the annotated source. Existing synchronization approaches, such as patch-based or diff-based merges, cannot bridge this gap without corrupting or discarding annotation markers, because they operate on line positions that no longer correspond between the two representations.

This thesis addresses that gap by introducing provenance-based synchronization. At derivation time, a per-line provenance map is recorded that stores, for each line in the projected variant, its exact origin index in the annotated source. At synchronization time, this map is used to translate diff opcodes (insertions, deletions, and replacements) from projected-line coordinates to annotated-source coordinates, applying each change precisely at its source position while leaving annotation markers and non-selected feature regions entirely untouched. The result is a structurally consistent round-trip: developers edit a clean variant, and their changes are lifted back into the annotated codebase without manual re-annotation.

III. Research Methodology

This thesis follows a Design Science Research (DSR) methodology [10]. The project is structured into three DSR cycles. Cycle 1 covers problem analysis and requirements identification, addressing RQ1. Cycle 2 covers the design and implementation of the artifact, addressing RQ2. Cycle 3 covers the evaluation of the artifact and addresses RQ3.

The motivation for the thesis is to address the limitations of traditional version control workflows for feature-specific development. Current Git workflows operate at the file and line level, which leads to loss of feature information and increased manual effort when working with feature-based code. To investigate how Git can better support feature-oriented development, the thesis addresses the following research questions.

A. RQ1: What requirements must a Git extension fulfill to support feature-based product derivation and synchronization while preserving feature traceability?

The existing feature-oriented Git prototype [1], [2] provides basic feature annotation and traceability but lacks dedicated support for two key operations: (1) deriving clean, ready-to-run product variants by removing non-selected feature code and annotation markers, and (2) safely synchronizing edits performed on such derived variants back into the annotated main codebase. The latter is particularly challenging because projected variants have a significantly different structure from the annotated source: different line numbers, missing markers, and excluded blocks.

This research question, therefore, focuses on the problem analysis and requirement identification phase. It aims to derive the functional and non-functional requirements needed to enable a practical, feature-aware workflow in Git, specifically addressing feature traceability, correct product derivation, structural synchronization of edits, divergence detection, and compatibility with standard Git operations. RQ1 is answered in Cycle 1 (Section IV).

B. RQ2: How can a feature-aware Git extension be designed and implemented to support feature-based product derivation and synchronization?

This question focuses on implementing the derivation and synchronization extension for the current feature-labeling Git prototype [1], [2]. The goal is to design a system in which developers can select a set of features, automatically derive a clean, ready-to-run product variant, make changes to that variant, and synchronize those changes back to the main codebase. The design must therefore ensure that feature-to-code mappings are preserved throughout both operations, and that the extension integrates naturally with standard Git workflows without replacing or disrupting them.

The question is answered in Cycle 2 (Section V) through three design components. First, the annotation mechanism, which defines how feature-to-code mappings are represented in source files. Second, the derivation engine, which produces a clean variant from a selected feature set using a stack-based annotation parser and provenance recording. Third, the synchronization mechanism, which translates edits made in the derived variant back to their correct positions in the annotated source using the provenance map.

C. RQ3: Do the proposed derivation and synchronization operations produce structurally correct outputs across the edit categories, annotation structures, and edge cases defined in the evaluation?

This research question is answered through a controlled correctness evaluation in Cycle 3 (Section VI). The evaluation verifies whether the core operations, being feature-based derivation and feature-aware synchronization, produce correct outputs across a representative set of scenarios. Correctness is treated as the primary evaluation objective because it is the necessary prerequisite for any practical workflow benefit: a synchronization mechanism that silently corrupts feature annotations or misplaces edits provides no value regardless of its efficiency.

The traditional Git workflow, in turn, provides context. It illustrates the problems the artifact is designed to eliminate,

such as manual marker preservation, unguided file-level integration, and the lack of divergence detection. Still, it is not used as an empirical comparison target, as this would require a user study beyond the scope of this thesis.

1) Evaluation Approach: A set of deterministic, predefined scenarios is constructed. Each scenario fully specifies an initial annotated source file, a selected feature set, an expected derived variant, a controlled modification applied to the variant, and an expected annotated source after synchronization. For each scenario, the artifact performs derivation and synchronization, and the resulting outcome is compared against the expected output by exact string comparison. The evaluation is deterministic and requires no user interaction; repeated executions yield identical results.

2) Evaluation Metrics: Three correctness properties are evaluated. In this study, a property is considered correct if the artifact's output matches the predefined expected outcome.

Product Derivation Correctness: Whether the derived variant contains exactly the lines belonging to the selected features, excludes non-selected code, and records the correct provenance mapping between projected and annotated source lines.

Synchronization Correctness: Whether modifications applied to a projected variant are transferred correctly back to the annotated source file after synchronization. The evaluation covers all supported edit categories: (1) equal-arity replacement ($k = m$), (2) unequal-arity replacement ($k \neq m$), (3) insertion, and (4) deletion.

Feature Structure Preservation: Whether annotation markers and non-selected feature content remain in their original placements after synchronization. This property specifically evaluates whether synchronization affects unrelated feature structure outside the modified provenance-mapped regions.

3) Scenario Selection and Criteria: The scenarios were designed to capture the main operations performed when editing derived variants: replacement, insertion, deletion, and unchanged lines, as well as important structural cases such as nested annotations, feature boundaries, and non-adjacent provenance. Each scenario was designed to test one specific property, and together they aim to cover all important combinations of edits and annotation structures within the thesis scope. The selection criteria are:

- Each scenario tests exactly one structural property.
- Scenarios collectively cover all four edit categories (replace $k = m$, replace $k \neq m$, insert, delete).
- At least one scenario tests nested annotations, non-adjacent provenance, file-level feature exclusion, and divergence detection.
- All inputs and expected outputs are fixed; no randomness is involved.

The evaluation aims to determine whether the proposed artifact produces correct outputs across the covered structural cases. Broader claims about workflow improvement or manual effort reduction would require a user study and are therefore out of scope.

IV. Cycle 1: Research and Planning

Cycle 1 is the first cycle of the project and focuses on research and planning. The goal of this phase is to analyze the problem in detail and derive a set of functional and non-functional requirements that guide the design and implementation of the artifact in the subsequent cycles. The cycle also establishes the scope of the project. The evaluation methodology is described under RQ3 in Section III.

A. Problem

Traditional Git workflows operate mainly at a file and line level and lack explicit support for feature-oriented development [8]. This leads to difficulties in:

- identifying which code belongs to what feature,
- deriving consistent product variants based on selected features,
- synchronizing feature-specific changes back to the code base.

Although current applications can improve feature traceability through labeling or annotations, they do not provide mechanisms for deriving or synchronizing feature-specific code in Git. This creates a gap where developers must manually:

- search and select relevant files for a feature,
- resolve structural conflicts caused by overlapping modifications between the projected variant and the annotated source.

To address these limitations, the Git extension must support traceability, product derivation, and synchronization. Based on the identified problems and research questions, the following requirements are derived.

B. Functional Requirements

1) R1: The system shall allow developers to retrieve all code related to a given feature when requested:

Motivation: Developers who need to work on feature-specific code should be able to do so without manually locating lines in files.

Relation to RQs: Supports RQ1 (requirements) and RQ2 (design of the derivation engine).

Verification: For each evaluation scenario, the variant produced by the derivation engine is compared by exact string comparison against the expected variant. R1 is satisfied if all derivation outputs match their expected outputs.

2) R2: The system shall allow developers to annotate code to a feature, so that feature-related code can clearly be identified:

Motivation: To distinguish feature-specific code and work on it in isolation, developers must be able to see what code belongs to which feature.

Relation to RQs: Supports RQ1 and RQ2 (design of the annotation mechanism).
Verification: Verified by confirming that annotated code is correctly identified and associated with the correct features during derivation and synchronization operations across all evaluation scenarios.

3) R3: The system shall apply changes made to feature code when committed back to the main repository, so that updates are not lost:

Motivation: Prevent loss of feature information during integration of edits made in a derived variant.
Relation to RQs: Supports RQ1 and RQ3 (synchronization correctness).
Verification: Verified by executing the synchronization scenarios and confirming that every edit category produces the expected annotated source by exact string comparison, and that synchronization is rejected when the annotated source has diverged since projection.

4) R4: The system shall work together with standard Git commands, so that normal workflows can still be used:

Motivation: Ensure the extension integrates with real-life Git workflows rather than replacing them.
Relation to RQs: Supports RQ1 and RQ2 (integration with Git).
Verification: Verified by confirming that the extension operates on real Git branches and commits, and that standard Git commands remain usable during and after derivation and synchronization operations.

C. Non-functional Requirements

1) NFR1 Simplicity: The system shall be implementable as a prototype within the scope of the thesis by limiting functionality to core features such as annotation, product derivation, and synchronization.

2) NFR2 Preservation: The system shall maintain correct associations between code and feature identifiers during derivation and synchronization by preserving annotation markers and the content of non-selected features in their original positions. This property is directly verifiable by exact string comparison of the annotated source before and after synchronization.

3) NFR3 Determinism: The system shall produce identical outputs for identical inputs across repeated invocations, ensuring that derivation and synchronization results are reproducible and suitable for automated evaluation.

4) NFR4 Scalability: The system shall scale approximately linearly with repository size for derivation and synchronization operations. This is to ensure that repositories containing increasing numbers of annotated files remain within practical runtime limits.

D. Scope

Given that the design and implementation of the artifact is conducted within the context of a bachelor's thesis, the scope of the project is deliberately constrained. This ensures that the resulting artifact can be developed, tested, and evaluated within the available time frame while still meeting the requirements. The scope keeps the implementation focused on the core functionality necessary to investigate feature-based product derivation and synchronization in Git, without extending into unnecessary or overly complex functionality.

The scope includes:

- feature-to-code mappings,
- product derivation based on feature selection,
- synchronization between variants and the main repository,
- correctness evaluation,
- scalability measurement on synthetic repositories of increasing size.

The scope excludes:

- user interface design,
- automated feature mapping (e.g., machine-learning-based feature inference),
- user studies and qualitative usability evaluation,
- empirical comparison against the traditional Git workflow.

E. Baseline: Traditional Git Workflow

The term "Traditional Git Workflow" refers to the current industry standard for how feature-oriented development is typically performed in Git. The baseline is described here to establish the context that motivates the artifact and to provide grounding for the evaluation results in Cycle 3.

In the traditional workflow, developers implement features using branches. When working on a specific feature, a developer typically:

1) creates or checks out a branch,
2) manually locates and modifies the relevant lines across one or more files,
3) commits the changes to the branch,
4) integrates changes back into the main branch using merges or cherry-picking.

This workflow operates at the file and line level. It provides no mechanism for isolating the code for a specific feature from other features in the same file. Therefore, it does not prevent accidental modification of annotation markers or adjacent feature blocks during editing, or detect divergence in the shared source between when a variant was derived and when changes are integrated back. These are precisely the properties the proposed artifact is designed to provide. Whether the artifact correctly provides them in the tested scenarios is examined in Cycle 3.

V. Cycle 2: Design and Implementation

A. Design Overview

This section presents the design of the proposed artifact: a feature-oriented extension to Git that enables feature-based product derivation and synchronization while preserving feature traceability. The artifact builds upon the existing feature-aware Git prototype [1], [2] and extends it with two new

capabilities: deriving a clean product variant from a selected feature set, and synchronizing edits from that variant back into the annotated source without disturbing unrelated code.

The implementation is structured around four concerns: annotation parsing, feature derivation, provenance recording, and synchronization. Among these, synchronization is the most technically challenging and is the core contribution of this thesis. The artifact is a Python command-line tool invoked as `git feature <command>`, which integrates directly with the Git working tree via GitPython.

B. Mapping Requirements to Design

Each functional requirement from Cycle 1 maps to a concrete design component:

R1: The retrieval of feature-based code is addressed by the product derivation engine described in Section V-D.

R2: Annotating code with features is addressed by the annotation mechanism described in Section V-E.

R3: Synchronizing changes back to the codebase is addressed by the provenance-based synchronization mechanism described in Section V-G.

R4: Integration with Git is addressed by exposing the artifact through a Git-aware CLI, described in Section V-H.

A fifth design element, provenance recording (Section V-F), is not tied to a single requirement but serves as the structural link that enables synchronization (R3).

C. System Architecture

The architecture has three main components:

Feature annotations: Embedded in source code, marking which lines belong to which feature.

A derivation engine: Scans tracked files, removes non-selected feature blocks, and writes the resulting code into a projection branch, while recording a provenance map.

A synchronization layer: Applies edits made on the projection branch back to the target branch using the provenance map.

A pre-commit hook on projection branches enforces edit isolation by preventing developers from accidentally committing changes to platform code. Git integration wraps all components, using GitPython for branch management, file checkout, commit creation, and repository state validation.

D. Feature-Based Product Derivation (R1)

What it does: Given a selected set of features, the derivation engine creates a new Git branch (the projection branch) containing only the relevant code, with all annotation markers removed.

The problem: A developer working on the `Login` feature should not have to read or reason about `Checkout` code. More importantly, if they are asked to edit a file that mixes both, any change they make risks unintentionally touching code that belongs to a different feature. The developer needs to work in a clean, focused environment.

The solution: The system walks every tracked file in the repository and applies a line-by-line projection. A stack tracks which feature annotations are currently open. Each line is either kept or discarded based on whether all features on the stack are in the selected set. Annotation marker lines themselves are always discarded. The result is written back to disk and committed to the new branch.

Consider the following annotated source file on the main branch:

```
# app.py -- annotated source, on the main branch

def setup():
    init_db()      # platform code, always
    ↪  present

# &begin[Login]
def login(user, pw):
    return auth(user, pw)
# &end[Login]

# &begin[Checkout]
def checkout(cart):
    return process(cart)
# &end[Checkout]
```

Projecting this file with `selected_features = {"Login"}` produces a clean variant on the new projection branch:

```
# project/Login branch -- app.py after
↪  derivation

def setup():
    init_db()

def login(user, pw):
    return auth(user, pw)
```

The `Checkout` block and all annotation markers are gone. The developer sees a clean file, ready to run.

Beyond line-level filtering, files can also be excluded entirely. A `.feature-map.json` configuration file maps glob patterns to features. Any file mapped exclusively to a non-selected feature is removed from the projection:

```
{
  "mappings": [
    { "pattern": "checkout/*", "feature":
    ↪  "Checkout" }
  ]
}
```

With this configuration, projecting with `{"Login"}` removes every file under `checkout/` from the projection branch.

E. Feature Annotation Mechanism (R2)

What it does: The annotation system lets developers mark which lines of a source file belong to which feature, directly

inside the file, without any external configuration (Section II provides full background on the syntax).

The problem: In a multi-feature codebase, a single source file often contains code for several independent features. Without a machine-readable boundary between those features, no tool can automatically extract or manipulate one feature's code in isolation.

The solution: The system uses the two inline markers `&begin[FeatureName]` and `&end[FeatureName]` from the existing prototype [1], [2]. Any line that matches a begin/end pair belongs to that feature. Lines outside any annotation block are platform code and belong to every product variant. The markers can appear inside any commented syntax, so no source language needs special support.

Three design decisions in this thesis shape how annotations are handled during derivation and synchronization:

1) Nesting: Annotations may be nested. A line enclosed within two annotations belongs to both features simultaneously and is included in the variant only when both are selected. This follows the union semantics of the existing prototype:

```
# &begin[Auth]
authenticate()
# &begin[OAuth]
oauth_handshake()    # included only when both
                     # Auth AND OAuth are
                     ↪  selected
# &end[OAuth]
# &end[Auth]
```

2) Defensive error handling: The parser handles malformed annotations without crashing. If a closing `&end[F]` marker appears while the top of the stack holds a different feature name, it is treated as regular content rather than as a structural command. This guarantees that a malformed marker can never silently corrupt a file by being misinterpreted.

3) File-level ownership: When an entire file belongs to a single feature, embedding per-block annotations would be redundant. The `.feature-map.json` configuration file (shown above in Section V-D) assigns whole-file ownership through glob patterns.

F. Provenance Recording

What it does: During derivation, the system records exactly which line in the annotated source each projected line came from. This record is stored in `.feature-provenance.json` and committed onto the projection branch.

The problem: After derivation, the projected file and the annotated source have different line counts. Annotation markers were removed, and entire feature blocks may have been stripped. If a developer later edits the projected file, there is no way to know which lines in the annotated source those edits correspond to, unless that mapping was recorded at the time of derivation.

The solution: During projection, alongside each kept line, its original index in the annotated source is recorded in a *provenance list*. A second parallel list records the feature stack active at that line: an empty stack indicates platform code; a non-empty stack indicates the line was inside a feature annotation block.

For the `app.py` example below, projecting with `{"Login"}` produces the provenance shown in Table 1:

TABLE I
PROVENANCE ENTRIES FOR THE LOGIN PROJECTION OF APP.PY.

Projected line	Content	Source index	Stack
0	`def setup():`	0	`[]`
1	`init_db()`	1	`[]`
2	`def login(...):`	3	`["Login"]`
3	`return auth(...)`	4	`["Login"]`

Source lines 2 (`&begin[Login]`), 5 (`&end[Login]`), and 6–9 (the entire `Checkout` block) are absent from the provenance list because they were stripped during derivation.

Both lists are stored per file in `.feature-provenance.json`, along with the SHA of the source commit at the time of projection:

```
{
  "version": 1,
  "source_sha": "a3f9c12e...",
  "files": {
    "app.py": {
      "lines":  [0, 1, 3, 4],
      "stacks": [[], [], ["Login"], ["Login"]]
    }
  }
}
```

The `source_sha` enables divergence detection during synchronization (Section V-G), and the `stacks` list enables edit isolation enforcement (Section V-I).

G. Synchronization (R3)

The challenge: Synchronizing edits from a derived variant back to the annotated source is not a standard merge problem. Conventional merge tools, including Git's own three-way merge, operate under the assumption that both sides of a merge are edits on top of a shared common ancestor, and that line positions across the two sides are structurally comparable. Neither assumption holds here. The projected variant is not a conflicting edit of the annotated source; it is a projection of the source, with annotation markers removed and entire non-selected feature blocks omitted. The two representations occupy different line spaces: a line at position `i` in the projected file does not correspond to line `i` in the annotated source. Applying a standard patch or diff-based merge directly would therefore either corrupt annotation markers or misplace edits into the wrong feature region.

The fundamental challenge is establishing a formal correspondence between these two incomparable line spaces before

any edit can be translated. Without such a correspondence, the concept of ”applying an edit at the correct location” is undefined. The provenance map recorded during derivation provides exactly this correspondence, making the translation from projected-line coordinates to annotated-source coordinates well-defined. Synchronization then reduces to a three-stage process by reconstructing the baseline, computing the edit delta, and applying the delta at the provenance-mapped positions, which the following sections describe.

Edit isolation: Beyond the structural challenge, synchronization must also respect a conceptual constraint. The codebase contains code for many features, but a developer editing the Login projection has, by definition, not seen and not reasoned about the code for Checkout, SpellCheck, or any other non-selected feature. It would therefore be incorrect for their edit to modify, reorder, or remove that code in any way. This is the edit isolation principle introduced by Walkingshaw and Ostermann in the context of projectional editing of variational software [11]: an edit made in one variant projection must not change the behavior or structure of any other variant. The artifact adopts this principle directly. Code belonging exclusively to non-selected features, and the annotation markers that delimit it, must be byte-for-byte identical in the annotated source before and after synchronization. The only changes that may appear in the annotated source are those derived from the developer's edits to lines that were part of the projected variant.

The solution: The synchronization mechanism enforces edit isolation by construction. The annotated source is treated as the authoritative file, and the algorithm modifies only lines whose provenance position falls within the developer's edit range. All other lines, including annotation markers and every line belonging to a non-selected feature, are passed through unchanged. Achieving this requires three stages: baseline reconstruction, diff computation, and provenance-guided patch application.

Stage 1: Baseline reconstruction: Before comparing anything, the algorithm reconstructs what the projected file looked like immediately after derivation, using the annotated source and the provenance map:

```
def reconstruct_baseline_lines(
        annotated_lines, provenance):
    return [annotated_lines[i]
            for i in provenance]
```

This picks exactly the lines from the annotated source that were included in the projection, in order. The result is identical to the projected file at the moment of the initial `git feature project` commit. Any edits the developer made on the projection branch are not yet reflected here; this is the starting point against which their edits will be diffed.

Stage 2: Diff computation: The developer's current version of the projected file is diffed against the reconstructed baseline using Python's `difflib.SequenceMatcher`. This produces a list of opcodes (operation codes): instructions describing how to transform the baseline into the developer's version. Each opcode is a five-tuple `(tag, i1, i2, j1, j2)` where `i1:i2` is a range in the baseline and `j1:j2` is the corresponding range in the projected file. The tag is one of `equal`, `replace`, `insert`, or `delete`. Only non-equal opcodes represent actual developer edits.

For example, if a developer renames `login` to `authenticate` in the projected file, the diff produces:

```
replace  i1=2  i2=3  j1=2  j2=3
```

This means: baseline line 2 (`def login(user, pw):`) should be replaced by projected line 2 (`def authenticate(user, pw):`). The provenance map shows that baseline line 2 corresponds to annotated source line 3, so that is the line that will be updated.

Stage 3: Provenance-guided patch application: The opcodes are applied to the *annotated source*, not to the baseline. For each opcode, the provenance map translates projected-file line indices into annotated-source line indices. The annotated source contains lines that do not appear in the provenance map at all, such as annotation markers and non-selected feature blocks, and those lines are never touched. Opcodes are processed in reverse order so that later (higher-index) modifications do not shift the line numbers of earlier positions before they are processed.

The gap problem and why per-line application matters: A critical edge case arises when a single diff opcode spans lines whose provenance indices are non-consecutive. This happens whenever annotation markers or a non-selected feature block are between two projected lines that the developer edited in a single change. A naive slice replacement of the form `output[src_start:src_end] = new_lines` would overwrite everything between `src_start` and `src_end`, including the annotation markers and non-selected feature code in the gap.

Consider the following annotated source with two features:

```
Line 0: shared1()
Line 1: # &begin[FeatureA]
Line 2: feature_a()
Line 3: # &end[FeatureA]
Line 4: # &begin[FeatureB]
Line 5: feature_b()
Line 6: # &end[FeatureB]
Line 7: shared2()
```

Projecting with `{"FeatureA"}` produces a three-line file:

```
Line 0: shared1()      <- source index 0
Line 1: feature_a()    <- source index 2
Line 2: shared2()      <- source index 7
```

The provenance map is `[0, 2, 7]`. Lines 1, 3, 4, 5, and 6 (the markers and the entire `FeatureB` block) are absent. If the developer edits both `shared1()` and `feature_a()` in a single change, the diff produces one replace opcode covering projected lines 0–1. A slice-based application would replace `output[0:7]`, deleting `FeatureB` and all four annotation markers. Instead, the per-line approach substitutes `output[0]` and `output[2]` individually, leaving lines 1, 3, 4, 5, and 6 completely untouched. The annotated source after sync becomes:

```
Line 0: shared1_updated()
Line 1: # &begin[FeatureA]
Line 2: feature_a_updated()
Line 3: # &end[FeatureA]
Line 4: # &begin[FeatureB]
Line 5: feature_b()   <- untouched
Line 6: # &end[FeatureB]
Line 7: shared2()
```

This per-line behavior is the central reason synchronization works correctly across non-adjacent provenance regions, which is one of the scenarios evaluated in Cycle 3.

Safety guards: Before the patch is applied, two safety checks are performed.

First, the algorithm checks whether the annotated source file has been modified on the target branch since the projection was made. This is detected by comparing the stored `source_sha` against the current `HEAD`. If the file has been modified, the sync is rejected, and the developer is asked to re-project before syncing back. Without this guard, the provenance indices could be misaligned with the current annotated source, and the patch would silently corrupt the file.

Second, the provenance indices are validated against the length of the annotated source and checked to be strictly increasing. If either check fails, the sync is rejected.

H. Integration with Git (R4)

What it does: The artifact integrates with standard Git workflows rather than replacing them. Developers familiar with Git can still use branches, commits, staging, and diffs as usual, while the extension adds feature-aware operations on top.

The problem: If a feature-oriented extension required developers to learn a new mental model or replace existing Git commands, adoption would be unrealistic. The extension must integrate naturally with the workflows developers already know.

The solution: The extension is exposed through Git-aware CLI commands that operate on real Git branches:

```
git feature project --include <feature> \
    --branch <name>

git feature project sync <projection-branch> \
    --target <branch>
```

Internally, the implementation uses GitPython to perform projection and synchronization through Git's existing object model, so every operation produces real branches and real commits. Developers can inspect projection branches with `git log`, view diffs with `git diff`, and interact with them like any other branch.

I. Edit Isolation on Projection Branches

What it does: A pre-commit hook prevents devel,opers from accidentally committing changes to platform code while working on a projection branch.

The problem: The projected file contains both feature code and platform code. A developer could accidentally modify a platform line, for example change `init_db()` to `init_db_v2()`. That edit would be syntactically valid, pass all tests, and commit without warning. When synchronization ran, the provenance map would propagate the change back to the annotated source, overwriting platform code that every other feature depends on.

The solution: The provenance stacks recorded during derivation identify which lines in the projected file are platform code (empty stack `[]`) and which are feature code (non-empty stack). Before every commit on a projection branch, the pre-commit hook diffs the staged file against the original projection baseline and checks the stack for each changed position. A `replace` or `delete` at a line whose stack is `[]` is a violation. An `insert` at a position where the preceding line has stack `[]` is also a violation.

For example, suppose a developer stages the following changes on the `Login` projection branch:

```
Line 1: init_db_v2()
        # stack=[] -> BLOCKED

Line 3: return auth_v2()
        # stack=["Login"] -> allowed
```

If a violation is found, the commit is aborted:

```
Error: Staged changes touch platform code
on a projection branch. Only code inside
feature annotation blocks may be modified.

app.py: cannot modify platform code
        at line 2 of the projection baseline
```

This ensures that the annotated source always remains the authoritative location for platform code changes and that projection branches stay focused on feature-level work.

J. Implementation Details

The artifact is implemented in Python 3.10+ using the following components.

Typer for CLI commands: The command-line interface is built using Typer, a Python library for building CLIs from function signatures and type hints. Each subcommand

(`project`, `sync`, `status`, etc.) is a plain Python function decorated with Typer, which handles argument parsing, help text generation, and error reporting. Typer supports the `git feature` invocation pattern via the `git-feature` entry point defined in `pyproject.toml`.

GitPython for repository operations: Reading tracked files, diffing branches, checking out files, staging changes, and creating commits are all handled through GitPython, which wraps the Git CLI and object model. GitPython was chosen over raw subprocess calls because it provides a structured API and raises typed exceptions, making error handling more predictable. For operations not covered by the high-level API, the `repo.git` escape hatch allows direct invocation of any Git command.

Annotation parsing: Feature annotation markers are parsed using two compiled regular expressions defined once at the module level:

```
_BEGIN_RE = re.compile(
    r".*&begin\[(?P<name>.*?)\].*")
_END_RE   = re.compile(
    r".*&end\[(?P<name>.*?)\].*")
```

The patterns use `.*` on both sides so the marker can appear anywhere on a line, inside any language's comment prefix (`#`, `//`, `--`), without language-specific parsing. The named capture group extracts the feature identifier directly. These two expressions are the single source of truth for what constitutes a valid annotation throughout the codebase.

K. Design Decisions and Trade-offs

Key trade-offs in Cycle 2:

1) Annotations vs. external metadata: Explicit code annotations were chosen to keep feature mappings visible and editable, at the cost of requiring developers to maintain annotations manually. The initial thesis proposal planned for automatic feature annotation through commit-history analysis, but this proved unreliable in practice, particularly for commits that touched multiple features. The implementation, therefore, relies on the manual annotations already supported by the original prototype. The trade-off is that developers must annotate their code by hand, but in exchange, they get precise, reliable mappings rather than approximate, error-prone inferences.

2) Block-level vs. line-level granularity: Annotations could in principle be applied at the line level, tagging each line with a feature identifier. The artifact instead uses block-level annotations, in which a contiguous region of code is associated with a feature via `&begin` / `&end` markers. Block-level annotations group feature code into clearly delimited regions, which mirrors how features are usually implemented in practice, as contiguous segments of related code. They also make the parser simpler and the file structure easier for developers to scan. The trade-off is reduced precision when feature code is scattered across non-contiguous lines, but in practice, this case is rare and can be handled with multiple annotated blocks.

VI. Cycle 3: Evaluation

This cycle evaluates the proposed artifact in two ways. The first being a controlled, scenario-based correctness study. This evaluation focuses on verifying whether the artifact produces structurally correct outputs for feature-based product derivation and synchronization across a representative set of structural cases. The second being a scalability evaluation, based on runtime measurements over repositories of increasing size. Correctness is treated as the primary evaluation objective because it is the necessary prerequisite for any practical workflow benefit: a synchronization mechanism that silently corrupts feature annotations or misplaces edits provides no value regardless of its efficiency.

It is important to note up front that this evaluation does not constitute a formal proof of correctness for all possible inputs. Rather, it provides evidence that the artifact works correctly in practice for the tested configurations and characterizes the conditions under which it has been verified.

A. Evaluation Setup

The evaluation is conducted using a dedicated Python test harness (`test/evalsuite.py`). The harness implements 10 deterministic scenarios, each specifying an initial annotated source file, a selected feature set, an expected projected variant, a controlled modification applied to the variant, and an expected annotated source after synchronization.

The harness executes each scenario in two phases. In the first phase, it invokes the derivation engine and compares the resulting variant with the expected output using exact string comparison. In the second phase, it applies the specified modification, invokes the synchronization engine, and compares the resulting annotated source against the expected output. Any discrepancy is reported as a unified diff.

The harness runs in two modes. The first mode targets a standalone reference implementation and verifies that the test specifications themselves are internally consistent. The second mode targets the real prototype implementation (`git_tool/materialization.py`). The evaluation is deterministic and requires no user interaction; repeated executions yield identical results.

Whitespace handling: The artifact preserves whitespace exactly as it appears in the annotated source. Exact string comparison, therefore, treats whitespace as significant. This means whitespace-only edits in a projected variant are reflected in the synchronized output. While this ensures that no silent whitespace corruption occurs, it also means that a developer's editor automatically reformatting indentation would propagate those reformatting changes back to the annotated source. This is acknowledged as a known limitation: the artifact does not attempt to distinguish semantic from non-semantic whitespace changes.

Annotation order: The order of annotation blocks in the source file is significant to the artifact. The provenance map is built by scanning the file from top to bottom, so the exact relative order of feature blocks is preserved. Reordering annotation blocks would produce a different provenance map and,

therefore, a different derived variant. This is by design: the artifact treats source order as part of the feature structure. The evaluation does not include a separate scenario for annotation reordering, since the artifact does not claim to support order-independent feature composition.

Scalability: The evaluation reported in Section VI-F characterizes how derivation and synchronization runtime grows with the number of annotated files on a single developers machine. It does not measure scaling in the dimensions of feature count, annotation density per file, or commit history depth, and it does not include concurrent or multi-user operation, or repositories backed by storage other than the local file system. The runtime values are therefore specific to the test machine and the synthetic file structure used; the scaling pattern with respect to file count, not the absolute numbers, is the reproducible finding.

B. Scenarios

The 10 scenarios are selected to cover both routine operations and structural edge cases arising from interactions between the provenance map and the annotation structure. Table II summarizes each scenario.

The scenarios cover all four edit categories: equal-arity replacement, unequal-arity replacement, insertion, and deletion. They also cover structural edge cases such as nested annotations, non-adjacent provenance, file-level exclusion, and divergence detection.

The scenarios intentionally focus on structural correctness at the line and block level. More complex cross-feature changes, such as renaming a variable that is used across multiple features or in variants that are not currently checked out, are not covered. Such changes require the developer to manually ensure consistency across all affected features, as the artifact operates only on the selected feature projection and has no visibility into other variants. This limitation is discussed further in Section VII.

Justification for Scenario 6 (empty markers): Scenario 6 verifies that when all lines inside a selected feature are deleted in a projected variant, the `&begin`/`&end` marker pair in the annotated source remains in place as an empty region. This is a deliberate design decision. Automatically removing the markers when their content is deleted would silently destroy feature structure information that the developer may still need: the empty block signals that the feature exists in the codebase but currently has no implementation. Equally important, the provenance map records source-line indices that exclude the marker lines. Hence, the synchronization mechanism has no way to remove markers without breaking the map's consistency for subsequent operations. The artifact, therefore, treats marker preservation as unconditional.

C. Metrics

The three metrics defined in Cycle 1 are evaluated as follows.

1) Product Derivation Correctness: For each scenario, the variant produced by the derivation engine is compared with the expected variant using exact string comparison. A scenario demonstrates correct behavior for this metric if and only if the two strings are identical.

Scenarios 1, 4, 8, and 9 primarily exercise derivation. Scenario 1 is the baseline case: a single feature is deselected, and its markers and content must be absent from the variant. Scenario 4 tests that a line enclosed within two nested annotations is excluded when the inner feature is not selected. Scenario 8 tests the derivation of a file in which the same feature appears in two non-adjacent blocks, producing a provenance map with two gaps. Scenario 9 tests file-level exclusion via `.feature-map.json`, in which an entire file is absent from the variant.

All 10 scenarios pass the derivation phase. The derived variants match their expected outputs in all cases, demonstrating that the stack-based annotation parser works correctly in practice for single features, nested features, multi-block features, and file-level exclusion across the tested configurations.

2) Synchronization Correctness: For each scenario that includes a modification (Scenarios 2–10), the annotated source produced by the synchronization mechanism is compared with the expected output using exact string comparison. A scenario demonstrates correct behavior if and only if the two strings are identical, or, in the case of Scenario 10, if the operation raises a divergence error before writing any output.

The four edit categories are each covered by at least one scenario.

Equal-arity replacement ($k = m$): Scenarios 2 and 7. In Scenario 2, a single return statement is changed. In Scenario 7, three lines on either side of and inside a feature boundary are each replaced individually. In both cases, each projected line maps 1-to-1 to its provenance position in the annotated source; marker lines between mapped positions are not part of the operation.

Unequal-arity replacement ($k \neq m$): Scenario 3. One projected line is replaced by three. The algorithm substitutes the first line at its provenance position and inserts the remaining two immediately after, placing them inside the feature region bounded by the surrounding markers.

Insertion: Scenario 5. A new line is appended after the last projected line of a selected feature. The insertion anchor is the provenance position of that last line, so the new line lands between the last selected line and the closing end marker inside the feature, not outside it.

Deletion: Scenario 6. Both lines inside a selected feature are deleted from the variant. The algorithm removes only the two mapped source lines. The begin and end markers, which were never part of the provenance map, remain in place as an empty feature region, as described in the scenario justification above.

All nine synchronization scenarios pass. Scenario 10 correctly raises a divergence error and writes nothing, confirming that the SHA check functions as intended within the tested configurations.

TABLE II
EVALUATION SCENARIOS.

#	Name	Modification Applied	Property Tested
1	Single-feature derivation	None	Markers and non-selected feature content are stripped, line provenance is correctly recorded
2	Single-line synchronization	One return value changed	A single edit maps to exactly one source line; annotation markers are preserved
3	Multi-line replace	One line expanded to three	New content is inserted at the exact correct position
4	Nested features	Edit inside outer feature	Inner unselected feature block is untouched; outer-feature edit is applied correctly
5	Insertion inside feature scope	New line appended after selected content	New line lands inside the `&begin`/`&end` pair
6	Deletion preserves empty markers	Both lines of a feature deleted	Only mapped source lines are removed; `&begin`/`&end` pair persists as an empty region
7	Edits at feature boundaries	Three simultaneous single-line edits at, inside, and after a feature region	Per-line substitution applied individually; no marker is shifted
8	Non-adjacent provenance	One line edited between two excluded feature blocks	Edit applies to the correct non-adjacent source line; surrounding excluded content is preserved
9	File-level feature mapping	Edit in the included file	Excluded file is absent from the variant but preserved byte-for-byte after sync
10	Invalid sync (diverged source)	Source modified after projection	SHA mismatch is detected; synchronization is rejected without modifying any file

3) Feature Structure Preservation: Feature structure preservation is not measured by a separate check but is implied by the synchronization correctness comparison. In every scenario, the expected annotated source contains all original markers and all non-selected feature content in their original positions, so a scenario can pass the string comparison only if both conditions hold. Preservation is therefore demonstrated for every scenario that passes synchronization correctness within the tested configurations.

Three scenarios specifically stress this property. Scenario 4 verifies that a nested unselected feature block survives an edit to its enclosing outer scope. Scenario 6 verifies that deleting all lines inside a feature does not remove the `&begin`/`&end` pair. Scenario 7 verifies that simultaneous edits on lines immediately adjacent to feature boundaries on both sides leave every marker in its original position. All three scenarios pass.

D. Comparison to Traditional Git

Table III reports the outcome of each scenario against the three metrics. D = derivation correctness, S = synchronization correctness, P = feature structure preservation.

TABLE III
RESULTS PER SCENARIO.

#	Scenario	D	S	P
1	Single-feature derivation	Pass	–	–
2	Single-line replacement	Pass	Pass	Pass
3	Multi-line replacement	Pass	Pass	Pass
4	Nested features	Pass	Pass	Pass
5	Insertion inside feature scope	Pass	Pass	Pass
6	Deletion preserves empty markers	Pass	Pass	Pass
7	Edits at feature boundaries	Pass	Pass	Pass
8	Non-adjacent provenance	Pass	Pass	Pass
9	File-level feature mapping	Pass	Pass	Pass
10	Diverged source	Pass	Pass	–

In the traditional Git workflow, the operations tested in these scenarios would require the developer to locate the relevant lines in the annotated source manually, apply changes while preserving surrounding markers by hand, and verify that no unrelated feature code was affected. There is no automated mechanism to prevent accidental marker removal or to detect that the source has been modified since the variant was derived. The scenarios in this evaluation represent precisely the situations where such manual operations are most error-prone: edits at feature boundaries (Scenario 7), edits that span provenance gaps (Scenario 8), and concurrent modification of the annotated source (Scenario 10). The fact that all 10 scenarios pass demonstrates that the proposed mechanism handles these situations correctly and automatically across the tested configurations.

E. Result

All 10 scenarios pass against both the reference implementation and the real prototype, verified by exact string comparison. These results provide evidence that the artifact works correctly in practice for the structural cases tested, though they do not constitute a formal proof of correctness for all possible inputs.

Product derivation works correctly in all 10 tested scenarios. The derivation engine correctly projects annotated sources across all tested configurations: single-feature, nested-feature, multi-block-feature, and file-level exclusion via `.feature-map.json`.

Synchronization works correctly across all four edit categories in Scenarios 2–9. Equal-arity edits are applied per-line at their exact provenance positions. Unequal-arity edits anchor new content at the first affected source position. Insertions land inside the enclosing feature scope. Deletions remove only mapped lines. Scenario 10 confirms that diverged-source detection works correctly: the operation is rejected before any file is written.

Feature structure preservation is demonstrated in all nine synchronization scenarios. Annotation markers and non-

selected feature content are preserved byte-for-byte in every tested case, including the three scenarios specifically designed to test boundary and nesting behavior.

These results directly answer RQ3: within the structural cases covered by the evaluation, the proposed approach produces correct derivation and synchronization outputs and preserves annotation structure throughout. Limitations regarding cross-feature changes and whitespace handling are discussed in section Section VII.

F. Scalability

To evaluate NFR4 (Scalability), the runtime of the derivation and synchronization operations was measured on synthetic repositories of increasing size. Each repository contained 10, 50, 100, or 500 annotated source files, with each file consisting of approximately 50 lines organized into two annotated feature blocks and surrounding platform code. The measurements were taken on a 2019 MacBook Pro (Intel Core i9-9980HK, 2.4 GHz, 8 cores) and repeated 5 times per configuration; the mean and standard deviation are reported in Table IV.

TABLE IV
RUNTIME OF DERIVATION AND SYNCHRONIZATION ACROSS REPOSITORY SIZES (MEAN ± STANDARD DEVIATION, 5 RUNS).

Files	Derivation (s)	Sync (s)	Derive ratio	Sync ratio
10	0.22 ± 0.00	0.51 ± 0.03	1.0×	1.0×
50	0.28 ± 0.01	1.53 ± 0.02	1.3×	3.0×
100	0.35 ± 0.01	2.82 ± 0.03	1.6×	5.5×
500	0.85 ± 0.02	13.39 ± 0.07	3.8×	26.2×

Both operations process each file independently, which predicts linear growth in the number of files. The synchronization results reflect this at scales above 50 files: scaling from 100 to 500 files (a factor of five) produces a $4.7\times$ increase in synchronization time. Derivation shows a smaller relative increase over the same range because its runtime is dominated by fixed-cost Git operations, such as branch creation and commits, which are performed once per invocation regardless of the file count. At very small repository sizes, this fixed overhead hides the per-file cost; at larger sizes, the per-file work becomes the dominant factor, and both operations approach linear scaling.

The standard deviations are consistently low, indicating that measurements are stable across runs. The absolute values are specific to the machine used and will differ on other hardware; the scaling pattern is the reproducible finding.

For repositories with a few hundred files, the measured runtime is within interactive limits. Nielsen identifies 10 seconds as the rough upper bound for keeping a user's attention focused on a single task [12]; estimating the outcome linearly from the 100-file measurement, a 300-file repository would take roughly nine seconds to synchronize, which sits at the edge of this bound. For substantially larger industrial codebases, the dominant cost is the per-file Git subprocess call performed once per file during synchronization. Batching these calls is an obvious optimization and is identified as a direction for future work.

VII. DISCUSSION

A. Results and Findings

The thesis investigated whether the proposed derivation and synchronization operations produce structurally correct outputs, while also preserving feature traceability within the defined scope. In turn, all 10 scenarios passed against both the reference implementation and the real prototype, confirmed by exact string comparison.

The derivation results confirm that the stack-based annotation parser correctly handles all tested structural cases, including single features, nested annotations, non-adjacent feature blocks, and file-level exclusion via `.feature-map.json`. The annotation parser successfully removes regions and markers for features that were not selected, while preserving the relevant features and provenance mappings. The provenance map is produced correctly in each case, which is a prerequisite for synchronization to function.

The synchronization mechanism also produces correct outputs for all types of edits, including equal-arity and unequal-arity replacements, insertions, and deletions. The mechanism correctly translates edits made in the projected variants back to their original location in the source files. Unrelated feature regions and annotation markers remained unchanged after synchronization, demonstrating that the artifact successfully preserves feature structures. The evaluation also showed that divergence detection works as expected. In Scenario 10, synchronization was blocked when the annotated source changed after projection.

The scalability measurement shows that both operations grow approximately linearly with the file count once the fixed costs of branch creation and commits are accounted for. The synchronization runtime remains within acceptable limits for repositories with up to several hundred files, with most of the scalability cost coming from repeated Git operations and per-file synchronization rather than from the annotation parsing itself.

To summarize, these results provide evidence that provenance-based synchronization is a viable mechanism for addressing the problem identified in the research gap: developers can now edit a clean derived variant and have those edits correctly synchronized back into the annotated source.

B. Future Work

Perhaps the most important extension would be support for automatic or semi-automatic feature annotation. The current approach relies on manual feature annotations. Developers have to tag the related feature code manually, and incorrect or incomplete annotations may lead to incorrect behavior. So while manual annotations improve transparency, they also introduce maintenance overhead. Techniques that could be considered include, for example, annotating code based on commit history analysis or machine-learning-based approaches that automatically identify feature-related code patterns.

The current sync-back guards against the case where the annotated source has been modified since the projection was

made. If such a divergence is detected, the operation is rejected with an error requiring the user to re-project. While this prevents data loss, it forces a full reprojection even when changes in the annotated source and the projected variant are in entirely separate regions of the file and could be merged automatically. Future work could replace this hard rejection with a three-way merge: using the reconstructed baseline (already available from the provenance map), the current annotated source, and the edited projected variant as the three inputs, a merge could be attempted automatically and only fall back to a conflict marker when the same region was touched on both sides.

Another point could be visualizing the tool through a dedicated user interface (UI). As of now, the artifact works via command-line arguments, which can be tedious to use. One idea is to visualize it in its own application or integrate it with an editor, such as Visual Studio Code. This could make the connection for real-life development even stronger by making the artifact easier to utilize.

Lastly, the artifact could be evaluated in a larger setting involving real developers in an industrial-scale setting with real-life repositories. The conducted evaluation focuses on deterministic and structural correctness scenarios, rather than real-world and large-scale development. Such a study could investigate whether the approach improves workflow efficiency, reduces manual effort, or how the extension affects the development of feature-oriented systems.

C. Validity Threats

1) Internal Validity Threats: Internal validity focuses on checking whether the work actually produced the outcome, rather than on factors outside of control or that are unmeasurable [13]. Essentially, can we attribute the result to the artifact, rather than to other factors. Since the evaluation is conducted using manually constructed scenarios, there is a risk that the chosen scenarios may influence the results. For example, the scenarios may not accurately represent how developers work in real projects.

Furthermore, the artifact builds upon an inherited prototype, which may influence the results, since some behavior stems from inherited functionality rather than mechanisms developed in this thesis. To reduce this threat, the evaluation relies on deterministic execution and predefined expected outcomes. The scenarios were also designed to cover all supported edit categories and edge cases, such as nested annotations, divergence detection, and provenance gaps.

2) Construct Validity Threats: Construct validity addresses whether the chosen metrics accurately measure the intended concepts [13]. In this study, correctness is measured by comparing the produced output with predefined expected results. Although the produced outcome passes the scenarios, it does not capture subjective experiences with the implemented extension. So, while the chosen evaluation provides a clear way of measuring correctness, it does not fully capture broader aspects, such as ease of use for developers or the levels of human satisfaction.

3) External Validity Threats: External validity concerns the generalization of the results [13]. The evaluation was conducted using synthetic repositories and controlled scenarios, rather than large industrial software product lines. As a result, findings may not generalize directly to actual, complex real-world environments, which often involve larger teams, greater repository complexity, and more advanced code. To mitigate these limitations, the scenarios were designed to reflect real-life development, including code modifications such as insertions, deletions, edits near feature boundaries, and nested annotations. Furthermore, the artifact was designed to work with, not instead of, standard Git mechanisms and workflows, which increases its applicability to real-world development.

VIII. Conclusion

This thesis investigated how Git can be extended to support feature-based product derivation and synchronization in software product lines. The motivation is the concrete gap in existing tooling. Although Git is the dominant version-control system in modern software development, it operates at the file and line level. It provides no native support for working with features as first-class units. Existing feature-oriented Git prototypes [1], [2] address labeling and tracking, but stop short of letting developers actually derive product variants and reintegrate edits made within them.

To close this gap, the thesis designed, implemented, and evaluated an extension that builds on top of an existing feature-annotation prototype and adds two new capabilities: feature-based product derivation, which automatically generates a clean, ready-to-run variant from a selected feature set, and provenance-based synchronization, which lifts edits made in that variant back to the annotated source while preserving annotation markers and unrelated feature content. The synchronization mechanism is the core contribution: it uses a per-line provenance map recorded at derivation time to translate developer edits between two structurally different representations of the same code without corrupting either.

The evaluation was conducted as a controlled scenario-based correctness study, complemented by a scalability measurement. All 10 scenarios passed against both the reference implementation and the real prototype, demonstrating that the artifact handles single features, nested annotations, non-adjacent feature blocks, file-level exclusion, all four edit categories, and divergence detection correctly within the tested configurations. The scalability measurement showed that both operations scale approximately linearly with repository size, remaining within interactive runtime limits for repositories of several hundred files.

The evaluation is bounded by what it tests. It does not constitute a formal proof of correctness for all possible inputs; it does not address cross-feature changes, such as renaming a symbol across non-selected variants; and it does not include a user study of developer experience. These limitations are made explicit in Cycle 3 and revisited in the future work section.

Within those bounds, the thesis demonstrates that feature-aware product derivation and synchronization can be inte-

grated into standard Git workflows in a structurally consistent and reliable way, without forcing developers to abandon the version-control concepts they already know. The result is a working round-trip workflow that prior research did not provide: developers can isolate a feature, edit it in a clean environment, and have their edits automatically lifted back into the annotated codebase, without manual re-annotation and without risk to the surrounding feature structure.

ACKNOWLEDGMENT

We want to begin by recognizing the efforts of all team members involved in this thesis. Daniel Norberg, Erik Gabrielsson, and Lilly Heier have all contributed equally to both the research process and the writing of this work.

We would furthermore like to express our sincere gratitude to our supervisor, Daniel Strüber, for his guidance, support, and valuable feedback throughout this period. He has been a great help to us, both in writing and implementing the thesis. His expertise and advice have been most beneficial, and we are grateful for his guidance.

Lastly, we would like to thank our examiner, Sven Peldszus, for his valuable feedback and constructive insights throughout our thesis work.

REFERENCES

[1] T.V. Röthemeyer, "Extending Git to Support Feature-Based Software Development," Ruhr-Universität Bochum, Bochum, Germany, 2024.

[2] H.T.K. Nguyen, "Improving and Evaluating the Git Extension Prototype 'Git with Features'," Ruhr-Universität Bochum, Bochum, Germany, 2025.

[3] D. Hinterreiter, L. Linsbauer, H. Prähofer, and P. Grünbacher, "Feature-oriented clone and pull operations for distributed development and evolution," *Software Quality Journal*, vol. 30, pp. 1039–1066, 2022.

[4] V. Kruskal, "A blast from the past: Using P-EDIT for multidimensional editing," IBM Thomas J. Watson Research Center, New York, USA, 2000.

[5] B. Andam, A. Burger, T. Berger, and M.R.V. Chaudron, "FLOrIDA: Feature LOcatIon DAshboard for Extracting and Visualizing Feature Traces," *ACM International Conference Proceeding Series*, vol. Part F126227, pp. 100–107, 2017.

[6] T. Schwarz, W. Mahmood, and T. Berger, "A Common Notation and Tool Support for Embedded Feature Annotations," *ACM International Conference Proceeding Series*, vol. Part F164402-B, pp. 5–8, 2020.

[7] M. Mukelabai, K. Hermann, T. Berger, and J.-P. Steghöfer, "FeatRacer: Locating Features Through Assisted Traceability," *IEEE Transactions on Software Engineering*, 2023.

[8] S. Apel, D. Batory, C. Kästner, and G. Saake, *Feature-Oriented Software Product Lines*. Berlin, Heidelberg, Germany: Springer, 2013.

[9] G.M. Ghodke and T. Chavan, "An Overview of Git," *International Journal of Scientific Research in Modern Science and Technology*, vol. 3, pp. 17–23, 2024.

[10] A.R. Hevner, S.T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[11] E. Walkingshaw and K. Ostermann, "Projectional Editing of Variational Software," *ACM SIGPLAN Notices*, vol. 50, no. 3, pp. 29–38, 2014.

[12] J. Nielsen, "Response Times: The 3 Important Limits," Nielsen Norman Group, 1993.

[13] R. Feldt and A. Magazinius, "Validity Threats in Empirical Software Engineering Research – An Initial Survey," Dept. of Computer Science and Engineering, Chalmers University of Technology, 2010.

[14] S. Stănciulescu, et al. "Concepts, operations, and feasibility of a projection-based variation control system." *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2016.